COMP1004 FINAL PORTFOLIO

Stargazing Conditions

Abstract

Application Design and Project Overview of an Astronomy based Weather Application tailored around the visibility of the night sky.

Seren James

CONTENTS

INTRODUCTION

The following report summarises the stages in the development of the Stargazing Conditions Weather Application. The focus of this project is to develop a Single Page Application that handles JSON input and output to and from a flat file and has a sufficient level of interactivity. This report gives overview of the project timeline, how tasks have been identified, assessed, managed, and implemented following the Software Development Lifecycle.

PROJECT PLAN

VISION

The primary objective of this application is to act as a stargazing aid and a tool for planning stargazing. The application should provide users with real-time, location specific weather forecasts that present meaningful information relevant to the overall stargazing conditions in the area.

BACKGROUND

Receiving real time weather data is possible through use of an API. The chosen API should supply data specific to factors affecting stargazing conditions. For favourable conditions, you need good transparency and seeing. Transparency refers to the opacity of the atmosphere whereas seeing is atmospheric turbulence. The main Factors affecting transparency and seeing are:

- Cloud Coverage
- Humidity
- Rainfall
- Dust
- Haze
- Light Pollution
- Winds

The application should provide an overall conditions rating based on the cumulation of these several variables. The greater the amount of these variables, the worse conditions will be. While generally rain is not good, light rain prior to stargazing can sometimes improve conditions by clearing the atmosphere of dust, haze, and other pollutants. As well as this, wind (that could distort the light waves) could improve conditions depending on the direction as Steve Richards (2019) highlights in Sky at Night Magazine: 'UK air will be clearer when the wind is blowing from the north than the south' because 'the air does not pick up pollutants from continental regions and the visibility will be crisp and clear.'

The most appropriate API for this project is Open weather API. Open weather is well documented and open source, meaning that there will be no issue using the API for a personal project. They also offer students 6 months access to the developer plan which grants almost all the features needed to build the application. Specifically, current weather data, daily 16-day forecast, and hourly 4-day forecast. Issues and constraints related to using Open Weather and the use of API's can be found in the Project Evaluation section of this document.

COMPETITOR RESEARCH

There are a couple applications that offer a similar service to the proposed application, they do not present this information in an easy-to-understand format. Sites I examined include:

- clearoutside.com
- meteoblue.com
- goodtostargaze.com

Initial assessments concluded that, while each provide a lot of useful information, the systems are overcomplicated to use and suffer from poor UI. As a result, they may be overwhelming for the average user.

To fully evaluate the sites, I tested each of them while stargazing. I found that data presented was unreliable and didn't match actual conditions. In addition, the sites were not understandable at first glance which was frustrating when trying to limit the time I spent looking at the light from my phone. My experience with meteoblue.com and goodtostargaze.com was greater in comparison due to the dark mode UI options. This allowed my eyes to readjust to the night sky a lot faster than when I used clearoutside.com. Although, the contrast of goodtostargaze.com made text a lot harder to read. Assessing the designs and features of competitors will aid the development of a successful product. A decomposition of the requirements related to this research can be found in the Requirements Analysis section of this document.

LEGAL AND ETHICAL

Guidelines and laws exist to ensure that websites uphold a certain standard. Copyright laws should be followed when sourcing assets. Unsplash.com has a range of images available for use. The Web Consortium Accessibility Guidelines highlight the ways in which developers can make websites more accessible.

SOFTWARE DEVELOPMENT LIFECYCLE

To successfully implement an effective solution, I have employed the Agile Methodology of the Software Development Lifecycle. The software development lifecycle is the process used by development teams when planning, designing, and implementing a new software application or system. The Agile SDLC is the iterative method for development. Breaking down the process of development into smaller chunks ensures that the system is thoroughly tested, refined, and optimized, resulting in a robust and high-quality digital solution. Agile also allows for greater flexibility to make changes to the system as the project progresses and requirements change.

This product will use the agile approach, rather than other methods such as the waterfall method. The waterfall method uses a linear development process where each stage must be completed before moving on which can delay development if an issue is encountered.

PRODUCT BACKLOG & SPRINTS

To keep track of the stages of development, I created a Trello board for active sprints and tasks on the backlog. To identify the tasks that should be defined on this board, I compiled a list of Functional and Non-Functional Requirements, as well as Key Performance Indicators. These were assessed throughout the project to ensure that all necessary items were eventually covered.

Each sprint was later decomposed into the six stages of the Agile SDLC method with one main task per sprint. This ensured a smooth development that did not overcomplicate the process. Planning stages involved assessing the Trello board and moving related tasks into the 'in progress' section. Analysis stages then involved identifying notable features and components of each task that were needed to complete them successfully.

Designs were then produced for relevant sprint tasks. These designs included UML diagrams to help decompose how users and systems would interact with the product as well as algorithmic designs. Most of the time in each sprint was spent implementing the task, for most tasks this meant coding the solution which then needed to be tested and refined to guarantee a robust product.

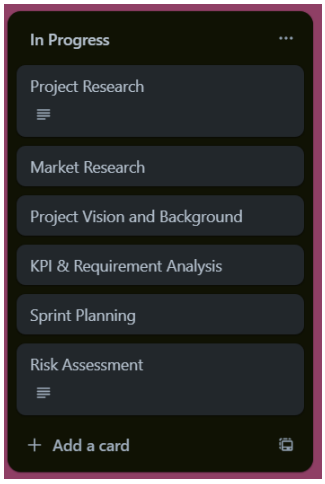
While each sprint followed its own iteration of the SDLC, so did the overall project The order sprints took place follows the SDLC with research, planning and analysis related items in the early stages of the project, followed by visual designs, code implementation, overall system testing and maintenance. This overviewing cycle repeated, at the start of each cycle, I re-evaluated the requirements of the system and made changes accordingly. For example, midway through development, changes were made to the UI designs to better tailor to user needs.

SPRINT PLANS

The following shows each sprint release as well as a brief description per sprint release.

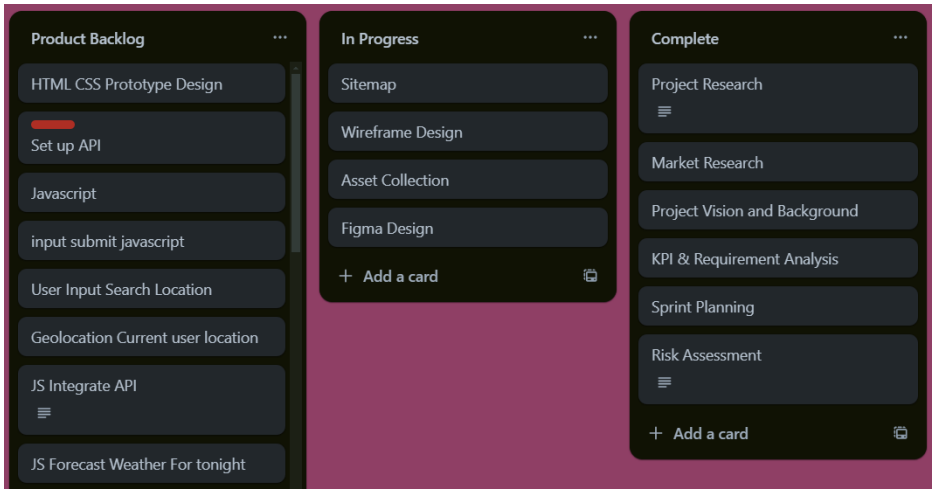
Sprint 1: 6/11/24

Early project stages saw Planning and Project Analysis.



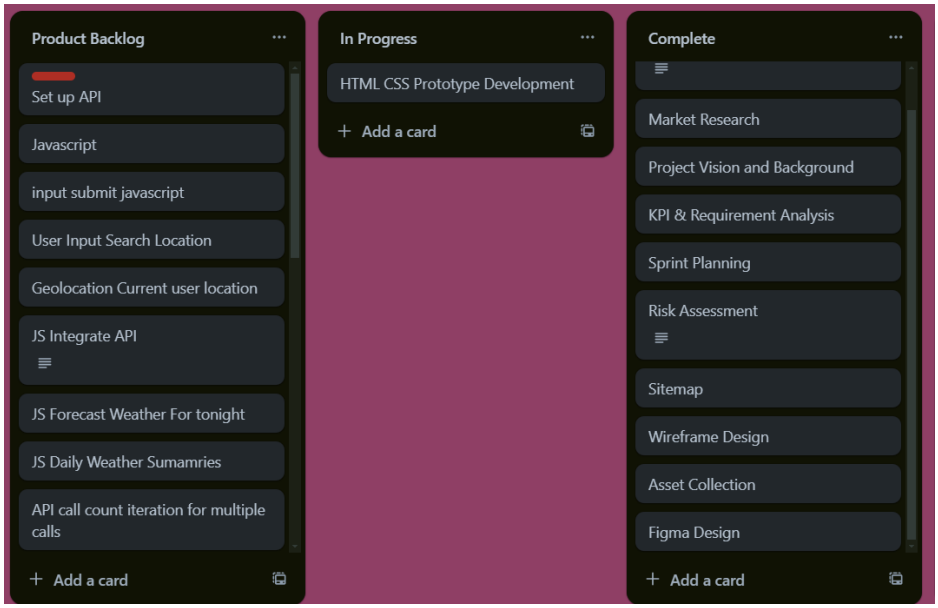
Sprint 2: 20/11/24

Sitemap and Initial Wireframing Designs produced. Figma mockup produced based on wireframe with assets found to suit the designs.



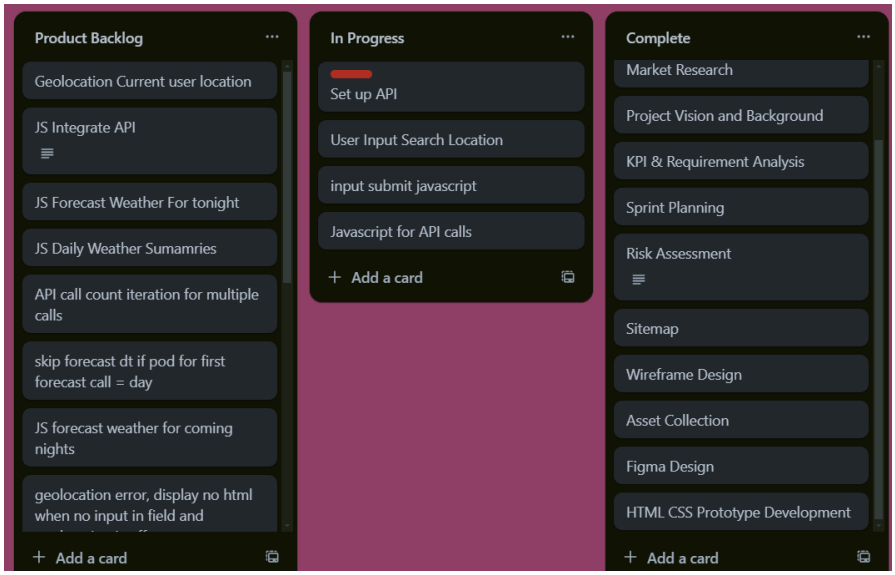
Sprint 3: 4/12/24

Basic Html and CSS structure for the site.



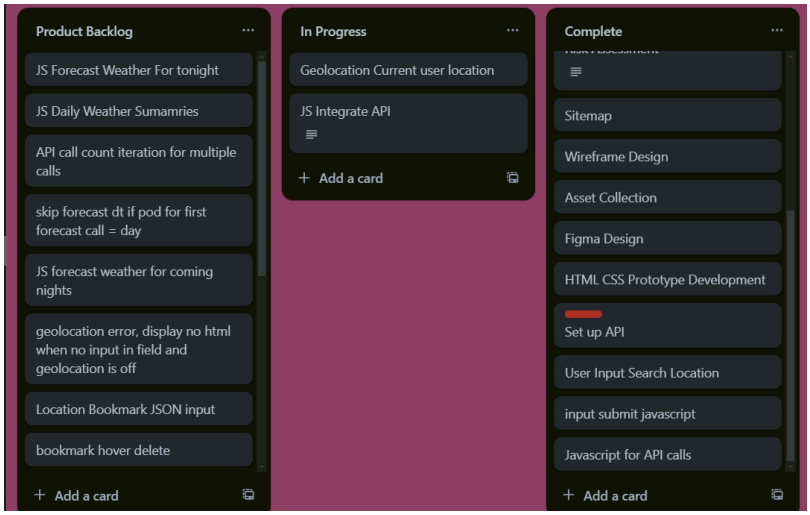
Sprint 4: 18/12/24

Weather API account created, applied for student developer plan. Developed JavaScript code for API calls. Prepared html with JavaScript interaction points e.g., Input Search



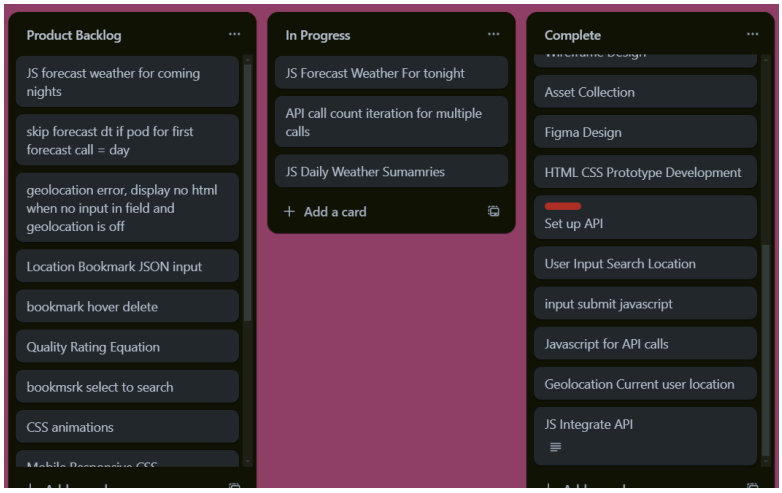
Sprint 5: 08/01/24

Integrated working JavaScript code to search for a location. Current Geolocation search feature began implementation.



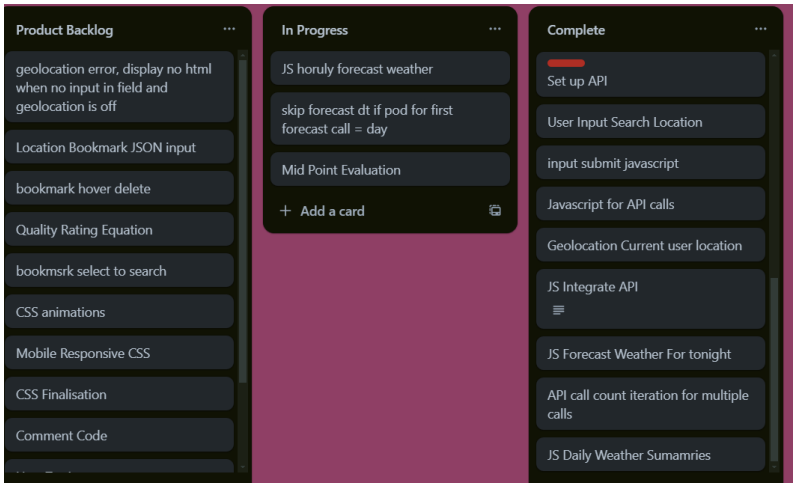
Sprint 6: 22/01/24

Tailored API calls to provide Daily Weather summaries for the night ahead and coming days using call count iteration.



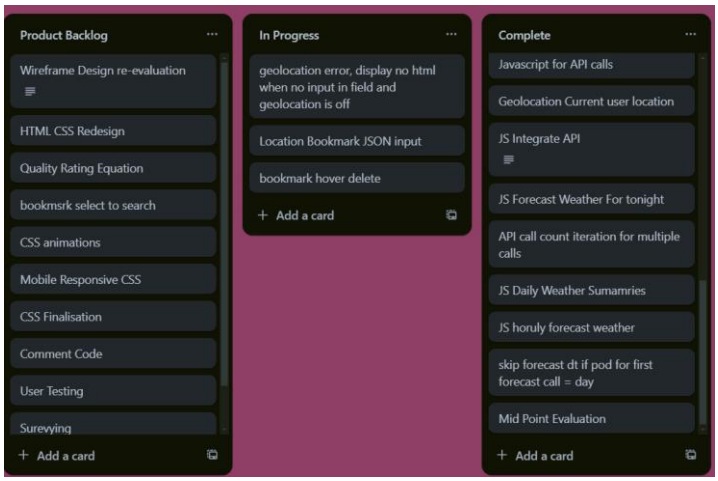
Sprint 7: 5/02/24

Reusing code for daily summaries, integrated code for hourly forecasts. These forecasts skip hours of daylight (pod=d). Project Midpoint Evaluation of designs and requirements.



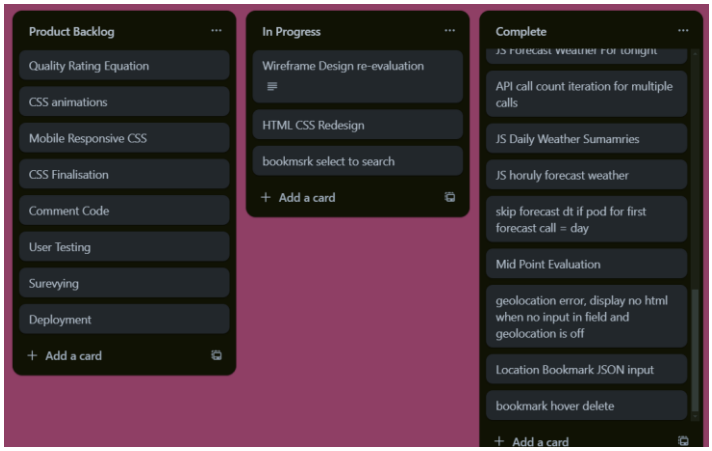
Sprint 8: 19/02/24

Following midpoint evaluation, wireframe revaluation and html/CSS redesign added to the project backlog. Ability to Save/bookmark favorite locations implemented and tested.



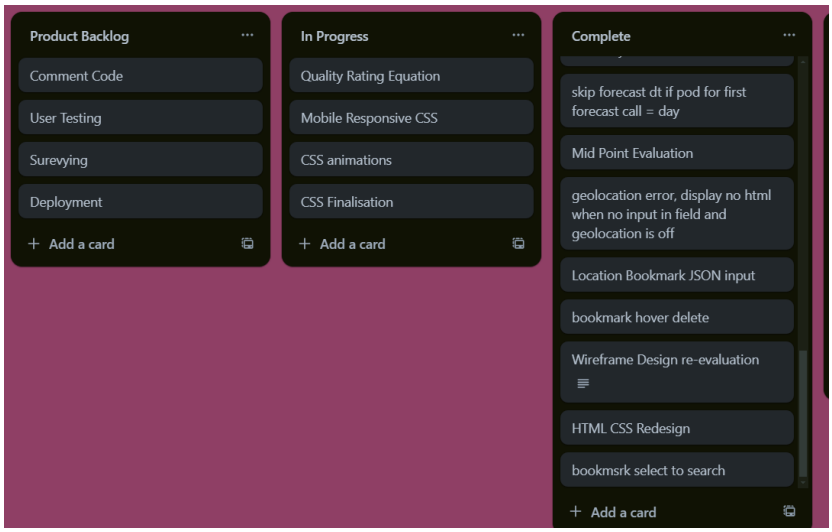
Sprint 9: 4/03/24

Wireframes updated, new Figma designs produced. HTML CSS amended to align with new designs. Added search functionality to Saved Locations



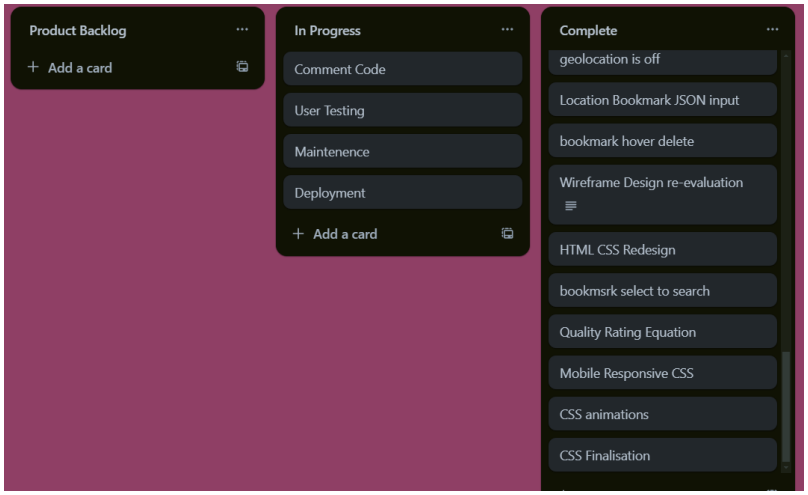
Sprint 10: 18/03/24

Calculation for stargazing quality rating with corresponding algorithm design produced and implemented. Added Mobile Responsive CSS styles, CSS animations and tidied up any issues CSS was causing.



Sprint 11: 1/04/24

Commented Code that had been missed. Beta tested the site with a small group, identified and fixed issues.



REQUIREMENTS ANALYSIS

FUNCTIONAL REQUIREMENTS

The specific requirements for the application are as follows:

- An overall conditions quality rating
- Reliable and accurate weather data
- Search for specific Location.
- Receive Location data based on current location.
- Save Locations for quick access.

NON-FUNCTIONAL REQUIREMENTS

The general requirements for the application are as follows:

Security

- Only access user location on request

Performance

- The site should have a high performance and response time.
- Should generate a high-performance lighthouse report.

Readability

- Use sizing to bring emphasis to important information.
- Colours should be overall darker to limit exposure to light when stargazing.
- Text should be clear and easy to read against the darker background.
- Content should be visually appealing and readable.
- Mobile Responsive to allow users on a range of devices to view the site.

Maintainability

- Code should be written with comments so that new developers can make changes in the future.

Scalability

- Able to handle an increase in network traffic.

Accessibility

- Keyboard Accessibility Options
- Alternative text

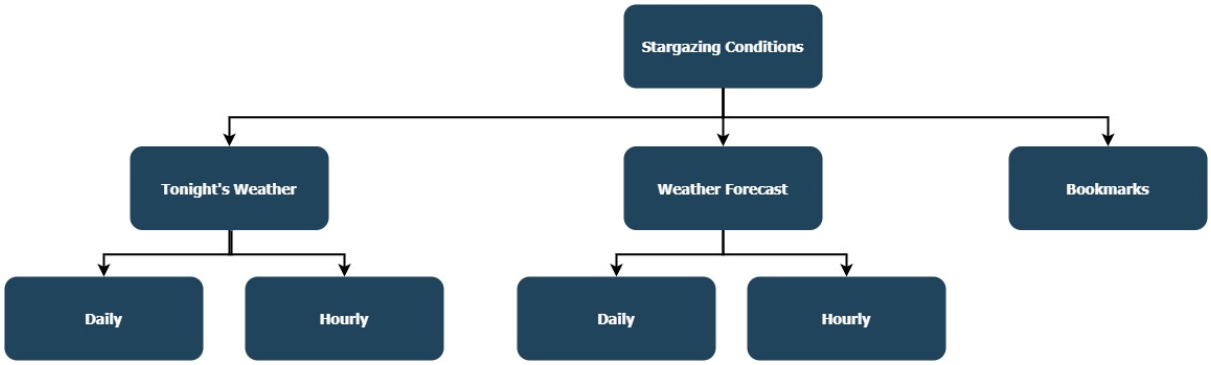
Feature	Priority	Justification
The site is Secure	High	Ensures a trustworthy service meaning users will be more likely to use the application
The site should perform well	High	As a basis, the site should run well
The site should be accessible	High	Part of the requirements for the website and WCAG

KPI'S

- Helpful tool for stargazing
- Short session durations
- Considerable number of API calls

VISUAL INTERFACE DESIGN

SITEMAP

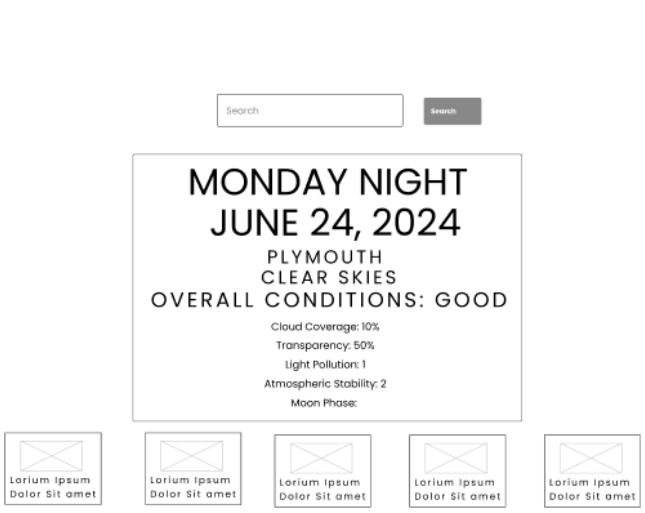


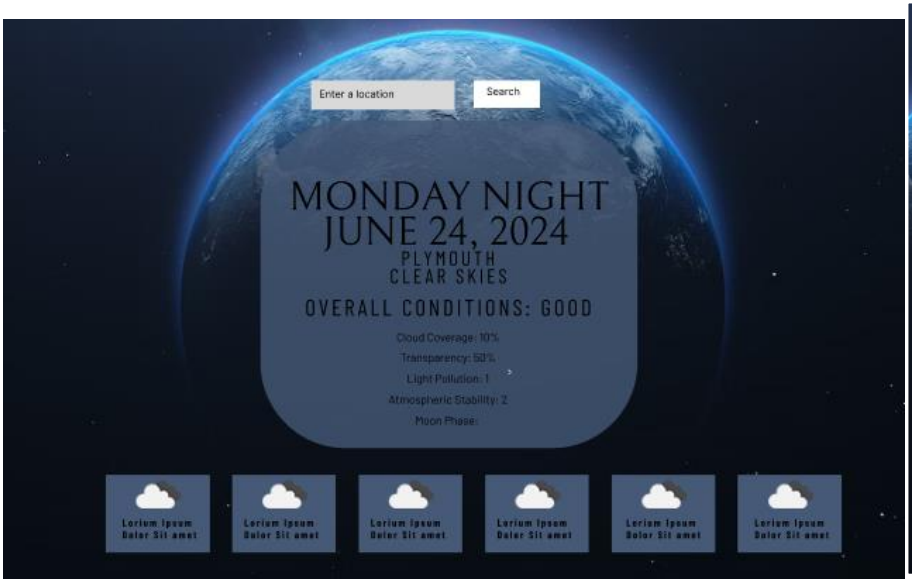
Sitemap shows the main content sections on the Single Page Application

WIREFRAMES AND UI FIGMA MOCKUPS

INITIAL DESIGN

Early Wireframe and UI designs provided a foundation for the structure of the site. However, these did not hit the UI quality target.



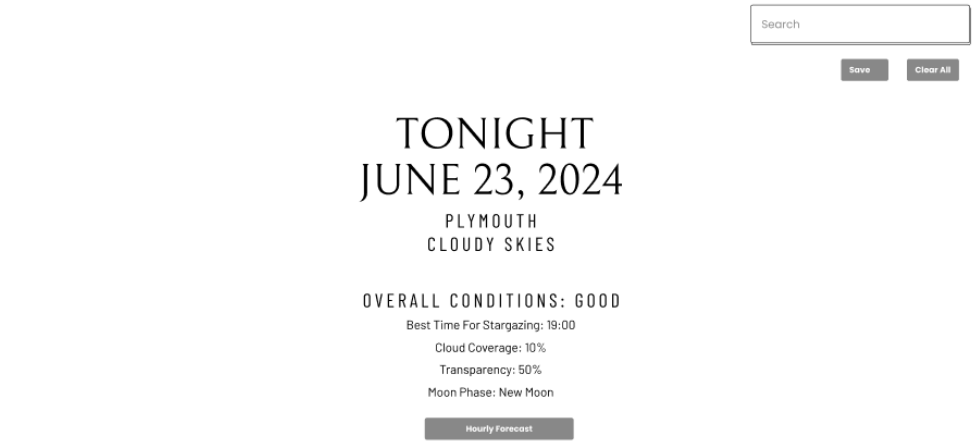


REDESIGNS

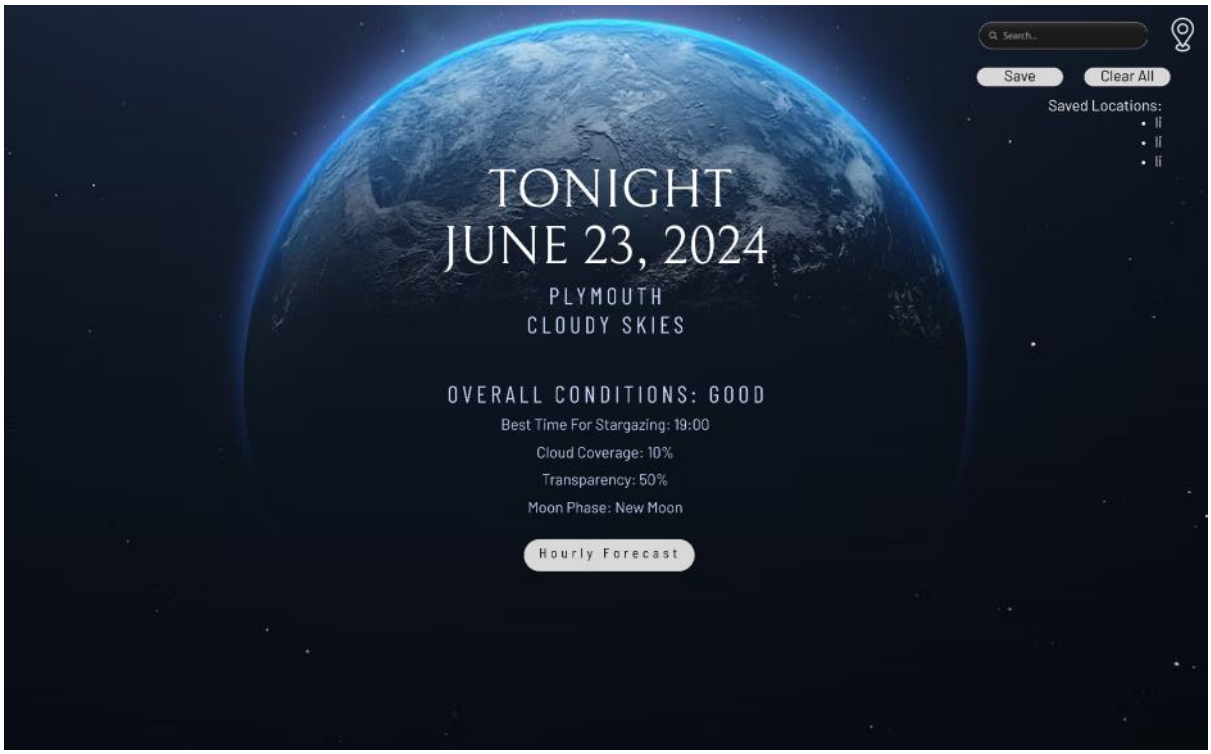
Following the midpoint evaluation of the project, two newer and similar wireframe and UI design options were produced and considered.

I considered hiding the search bar and displaying the location after the input had been filled. This idea was later disregarded as the location is already displayed on the page and would reduce the ease of use.

Ultimately, the decided wireframe design was the following:


Search
Save
Clear All
TONIGHT
JUNE 23, 2024
PLYMOUTH
CLOUDY SKIES
OVERALL CONDITIONS: GOOD
Best Time For Stargazing: 19:00
Cloud Coverage: 10%
Transparency: 50%
Moon Phase: New Moon
Hourly Forecast


MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY


Search...
Save
Clear All
Saved Locations:
TONIGHT
JUNE 23, 2024
PLYMOUTH
CLOUDY SKIES
OVERALL CONDITIONS: GOOD
Best Time For Stargazing: 19:00
Cloud Coverage: 10%
Transparency: 50%
Moon Phase: New Moon
Hourly Forecast


MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY

Mobile


Search...
TONIGHT
JUNE 23, 2024
PLYMOUTH
CLOUDY SKIES
OVERALL CONDITIONS: GOOD
Best Time For Stargazing: 19:00
Cloud Coverage: 10%
Transparency: 50%
Moon Phase: New Moon
MONDAY NIGHT
JUNE 24, 2024
PLYMOUTH
CLEAR SKIES
OVERALL CONDITIONS: GOOD
Cloud Coverage: 10%
Transparency: 50%
Light Pollution: 1
Atmospheric Stability: 2
Moon Phase:
25/6/24
26/6/24
27/6/24

DATA REQUIREMENTS & DESIGNS

UML DIAGRAMS

USER STORIES

As a user planning when to go stargazing I want to access weather data for the coming days so that I can choose the best night to go stargazing for my location

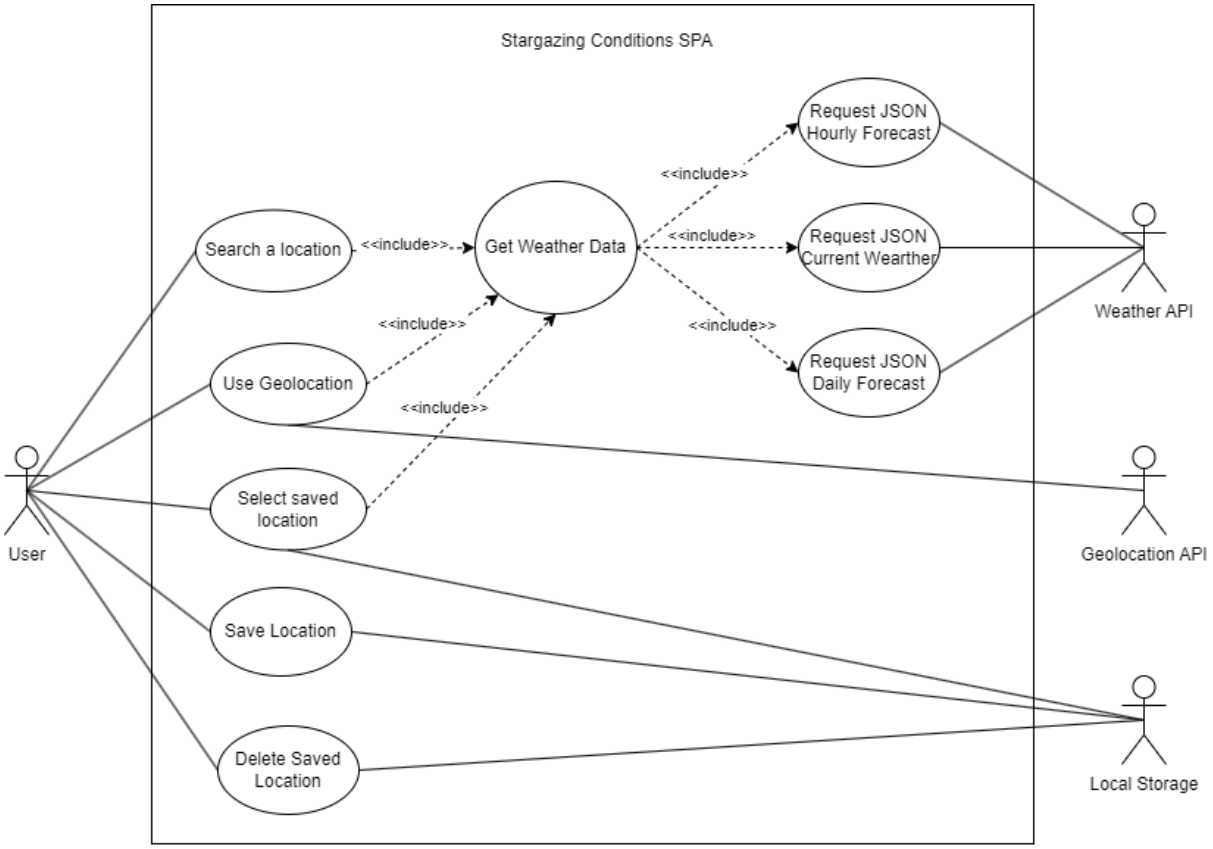
As a user planning a night stargazing tonight, I want to access the hourly weather data for tonight so that I can figure out the best hours to stargaze for my location.

As a user currently stargazing under poor conditions tonight, I want to access the hourly weather data for my location tonight so that I can figure out when good conditions will return.

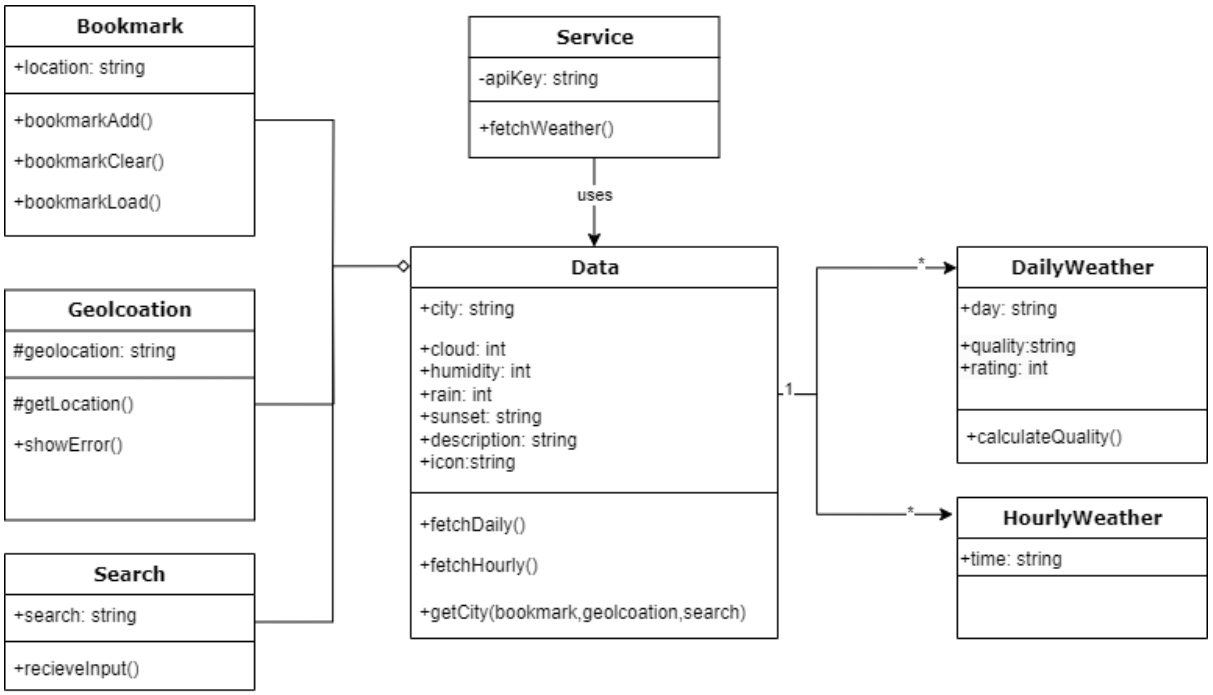
As a stargazer planning to travel I want to find tonight's current conditions for a remote location so that I can find the best location.

As a user who is frequent stargazer, I want to quickly access locations that I frequently search for so that I can quickly find the best stargazing locations for the night.

USE CASE DIAGRAM



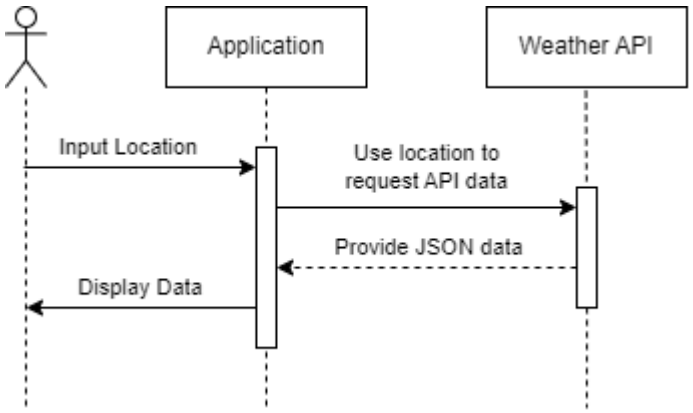
CLASS DIAGRAM



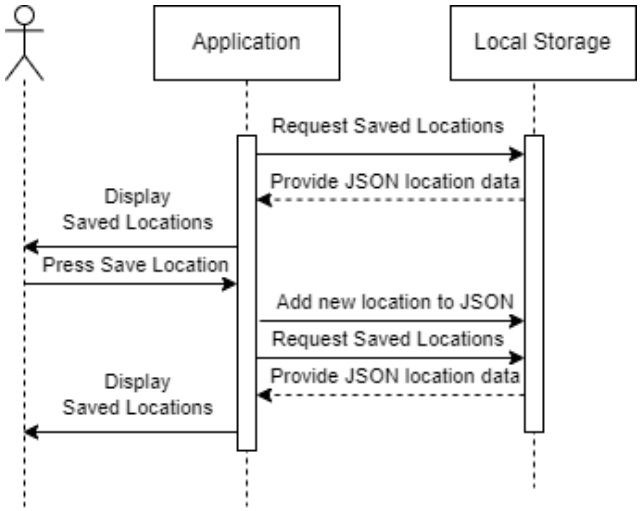
ARCHITECTURE

SEQUENCE INTERACTION DIAGRAMS

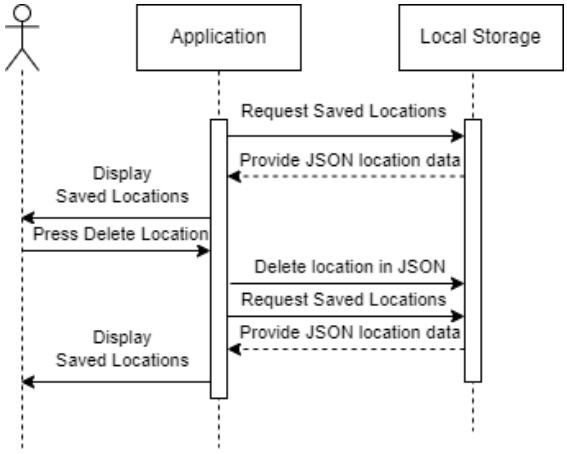
Representation of a user accessing weather data via search input:



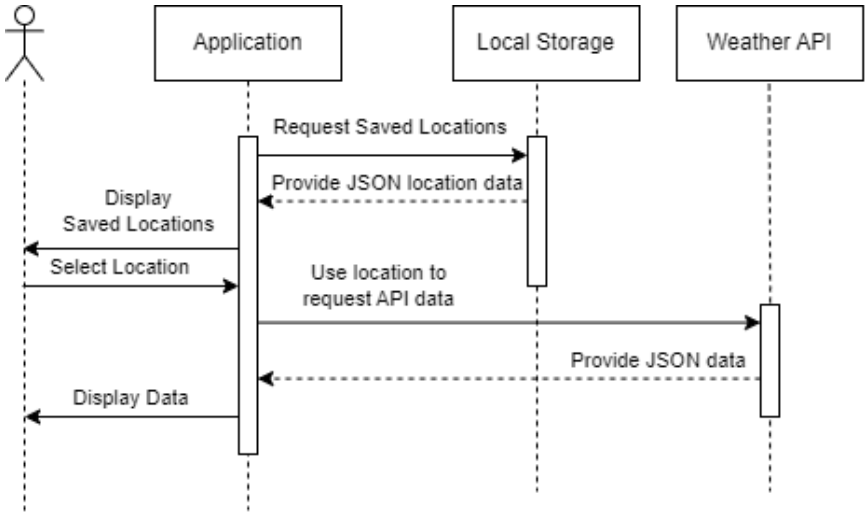
Representation of a user Saving a location:



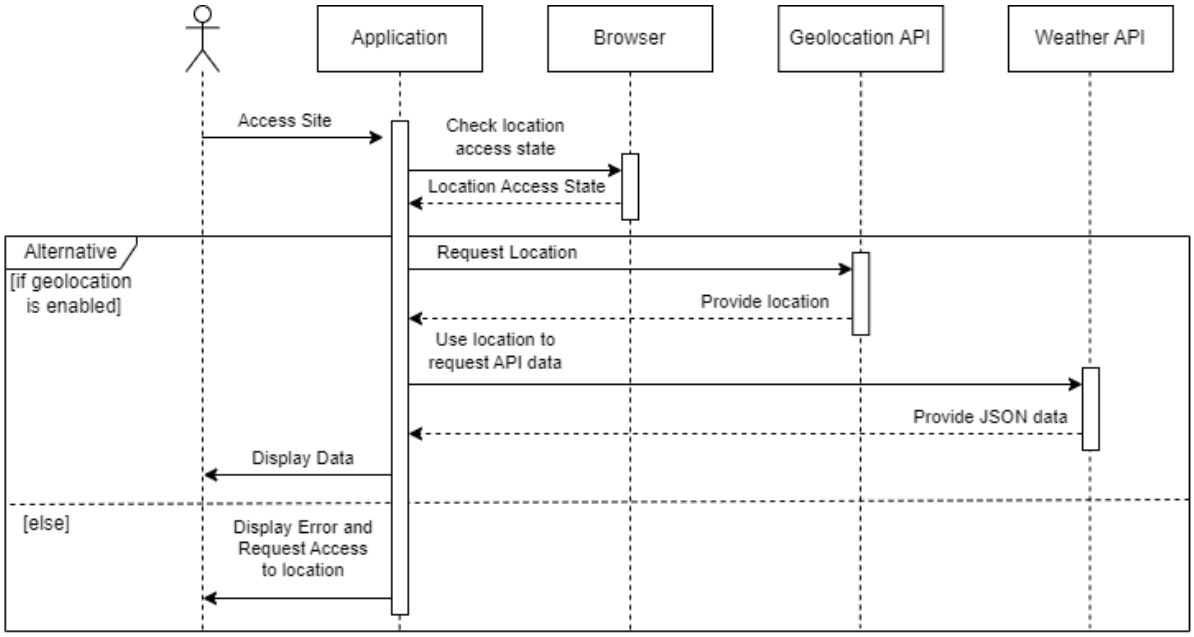
Representation of a user Deleting a location:



Representation of a user accessing weather data via saved locations:

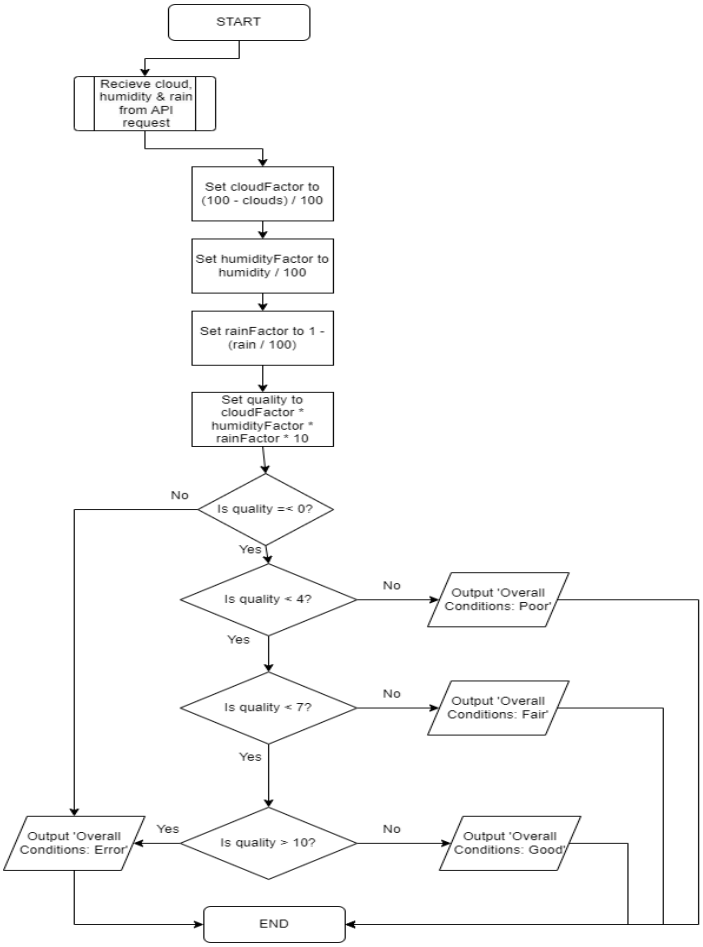


Representation of a user accessing weather data via their current location:



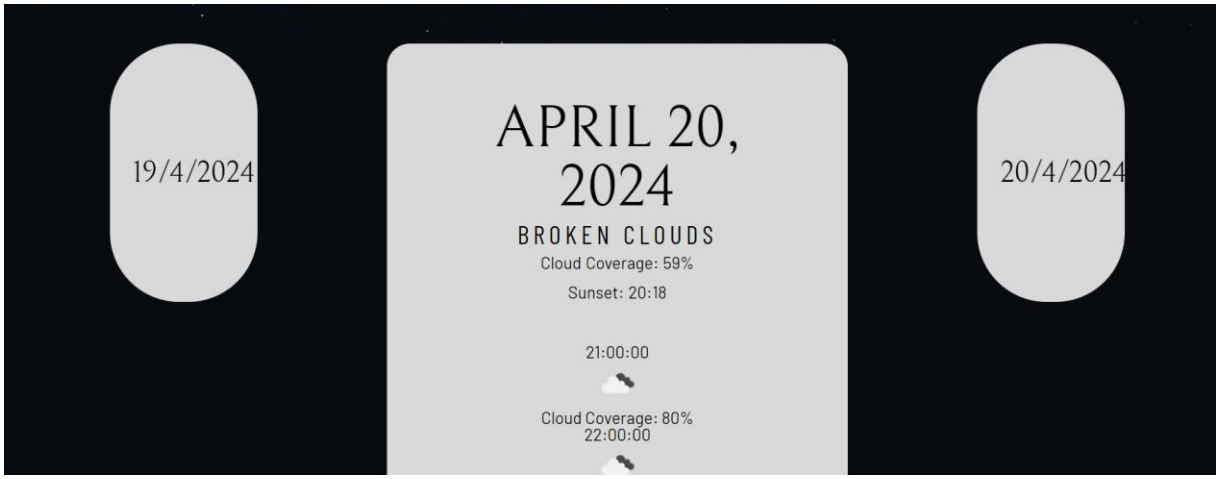
ALGORITHM DESIGN

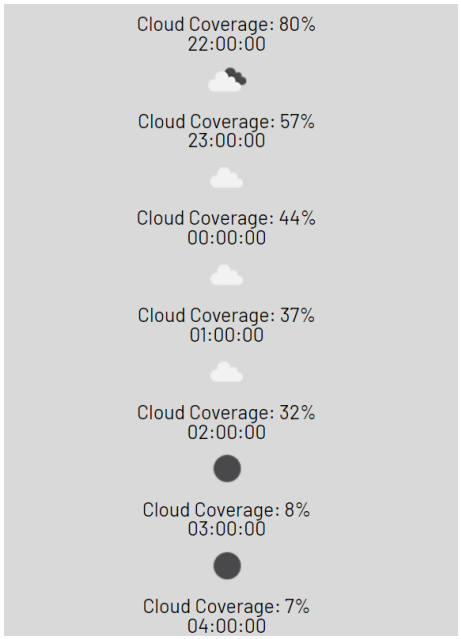
This Algorithm design demonstrates how the stargazing conditions calculation function works within the code.



IMPLEMENTATION

Screenshots of the functional product:





PROJECT EVALUATION

Overall, the Stargazing Conditions application has achieved its main requirements in providing clear, reliable weather data to aid stargazers.

The project has adapted slightly from the initial plan with the introduction of a bookmarking feature that allows users to save locations to the browser's local storage in JSON format.

It was difficult to find an API that provided data relevant to stargazing conditions. Despite it not having all the factors required to give an accurate quality rating, the data provided by Open Weather gives an adequate solution to the stargazing quality rating. Since the API did not provide enough data to justify implementing a toggle feature, this idea was abandoned. However, I would like to see that it is implemented in the future along with other sets of data such as light pollution. Further development of the site could see it become a multiple page application with one page being a light pollution map. It would not make sense to add this to the current page as it would decrease performance and overcomplicate the UI presentation.

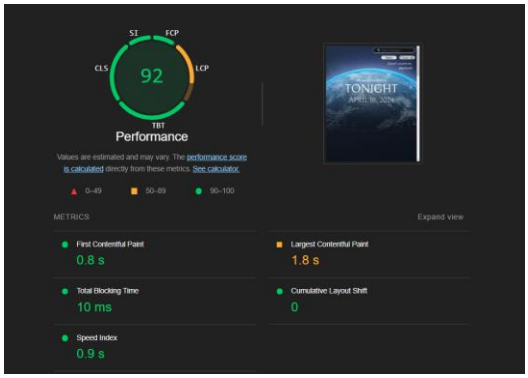
In terms of changes to the initial proposals of features, the percentage rating for the quality of conditions was changed to give a score out of ten to provide users with a simplistic rating.

The sites functionality is as expected, and all Functional Requirements have been met.

While accessibility features are limited to those provided by the browser, the site does ensure that content is sized appropriately and is clear and easy to read. Accessibility was a major consideration when developing the project, not only to meet legal and regulatory requirements but also to meet the individual requirement of having a darker UI. Midway through the project, the designs were completely overhauled to better align with these requirements.

The security of the application is to standard with Data Protection Laws. The site does not collect any unnecessary information; the only data stored is the saved locations provided by the user which can be removed at any time. Geolocation also operates per request. The site requests access to user location when they first access the site, and this data is forgotten as soon as the API request has been fulfilled.

Performance of the site was limited by the time it took for Open Weather to send API data. As demonstrated in the sequence diagram - after loading the page with geolocation, the site would need to send a request to the geolocation API which then needed to be processed by the application before requesting the data from the weather API. This increased the call time for the API by 2 seconds. However, load times have been hidden using CSS fade in animations. Lighthouse generates an outstanding performance report when analysing the site.



Maintainability of the site has been made easier through effective commenting but as the API receives updates, the current code may not work for API calls. This means the system may need to be maintained and

updated too. Additionally, the student developer plan which currently provides access to most of the weather data will run out in time and the product will not be able to sustain itself unless it generates income. Further research and development could involve shifting to an alternative source of data than API's.

GITHUB REPO

https://github.com/s0up4brains/COMP1004SPA

REFERENCES

First Light Optics Ltd (n.d.). Clear Outside v1.0 - International Weather Forecasts For Astronomers. [online] clearoutside.com. Available at: https://clearoutside.com.

Lloyd, M. (n.d.). Good To Stargaze. [online] www.goodtostargaze.com. Available at: https://www.goodtostargaze.com/.

meteoblue (n.d.). Weather. [online] meteoblue. Available at: https://www.meteoblue.com.

Ferreira, A. and Medium (2023). *Agile Software Development Lifecycle*. [Web Article] *Medium.com*. Available at: https://medium.com/@argferreira1/deep-dive-the-software-development-life-cycle-sdlc-in-agile-environments-6b245bbea5ab.

Steve Richards (2019). Astronomy weather forecasts: predicting the weather for stargazing. [online] www.skyatnightmagazine.com. Available at: https://www.skyatnightmagazine.com/advice/skills/how-predict-weather-forecast-astronomy-stargazing [Accessed 9 Dec. 2023].